PROJECT REPORT

Intelligent Information Retrieval and Knowledge Management using Multimodal AI

Submitted by

16500123002 – Tapaban Ray

16500123024 – Saptaparna Pati

16500123036 – Swarnava Maity

16500123038 – Ahyan Kamal

Group: [GROUP NUMBER]

Final Year – 7th Semester

[MONTH, YEAR]

Submitted for the partial fulfillment of the requirements for the degree of

Bachelor of Technology in

Computer Science and Engineering

Techno Main Salt Lake

EM 4/1, Salt Lake, Sector V, Kolkata – 700 091

Department of Computer Science and Engineering

Techno Main Salt Lake

Kolkata – 700 091

West Bengal, India

APPROVAL

This is to certify that the project entitled "Intelligent Information Retrieval and Knowledge Management using Multimodal AI" prepared by **Tapaban Ray (16500123002)**, **Saptaparna Pati (16500123024)**, **Swarnava Maity (16500123036)** and **Ahyan Kamal (16500123038)** be accepted in partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering.

It is to be understood that by this approval, the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn thereof, but approves the report only for the purpose for which it has been submitted.

(Signature of the Internal Guide) (Signature of the HOD)

(Signature of the External Examiner)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Techno Main Salt Lake

Kolkata – 700 091

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide in the Department of Computer Science and Engineering, Techno Main Salt Lake, for providing us with continuous guidance, valuable suggestions and encouragement throughout the development of this project.

We are extremely thankful for the keen interest our guide took in advising us, for providing useful technical directions, reference materials and constructive feedback at various stages of the project. The guidance and support received during the analysis, design and development phases significantly contributed to the successful completion of this work.

We would also like to express our gratitude to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering for providing us with the academic foundation, technical knowledge and resources required for undertaking this project.

We are thankful to all the non-teaching staff for their cooperation and support during the course of our project work.

Last but not least, we would like to thank our friends and family members for their continuous encouragement, patience and support throughout the development of this project.

Place: Techno Main Salt Lake

Date: _____

Tapaban Ray

Saptaparna Pati

Swarnava Maity

Ahyan Kamal

Contents

APPROVAL **i**

ACKNOWLEDGEMENT **iii**

Abstract **viii**

1 Introduction **ix**
- 1.1 Background . . . ix
- 1.2 Motivation . . . ix
- 1.3 Problem Statement . . . x
- 1.4 Technical Domain . . . x
- 1.5 Business Domain . . . xi
- 1.6 Glossary . . . xi

2 Related Studies **xii**
- 2.1 Introduction . . . xii
- 2.2 Traditional Information Retrieval . . . xii
- 2.3 Neural Embeddings . . . xii
- 2.4 Transformer Architecture . . . xii
- 2.5 Vector Databases . . . xii
- 2.6 Retrieval-Augmented Generation . . . xiii
- 2.7 Multimodal Information Retrieval . . . xiii

3 Problem Definition and Preliminaries **xiv**
- 3.1 Problem Definition . . . xiv
- 3.2 Scope . . . xiv
- 3.3 Project Boundaries . . . xiv
- 3.4 Preliminaries . . . xiv
- 3.5 Similarity Calculation . . . xv

4 Proposed Solution **xvi**
- 4.1 Overview . . . xvi
- 4.2 System Workflow . . . xvi

4.3 Document Ingestion Pipeline xvi
4.4 Semantic Search Pipeline xvi
4.5 Document Chat Pipeline xvii
4.6 Major Contributions xvii

5 Project Planning **xviii**
5.1 Development Methodology xviii
5.2 Major Project Phases xviii
5.3 Software Life Cycle Model xviii
5.4 Estimated Cost xviii

6 Requirement Analysis **xx**
6.1 Functional Requirements xx
6.1.1 User Registration and Authentication xx
6.1.2 Document Upload xx
6.1.3 Document Processing xx
6.1.4 Embedding Generation xx
6.1.5 Semantic Search xx
6.1.6 Document Chat xx
6.1.7 Search Result Ranking xx
6.2 Non-Functional Requirements xx
6.3 Requirement Matrix xxi
6.4 Requirement Elaboration xxii
6.4.1 FR-01 User Authentication xxii
6.4.2 FR-02 File Upload xxii
6.4.3 FR-03 Content Extraction xxii
6.4.4 FR-04 Embedding Generation xxii
6.4.5 FR-05 Vector Storage xxii
6.4.6 FR-06 Semantic Search xxii
6.4.7 FR-07 Top-K Retrieval xxii
6.4.8 FR-08 Document Chat xxii
6.4.9 FR-09 Retrieval-Augmented Generation xxii
6.4.10 NFR-01 Security xxii

7 Design **xxiii**
7.1 Technical Environment xxiii
7.1.1 Hardware Requirements xxiii
7.1.2 Software Environment xxiii
7.2 Detailed Design xxiii
7.2.1 System Architecture xxiii

7.2.2 Document Ingestion Module xxiv
7.2.3 Embedding Module xxiv
7.2.4 Semantic Search Module xxv
7.2.5 Retrieval Module xxv
7.2.6 RAG Module xxv
7.2.7 Database Design xxv

8 Implementation **xxvii**
8.1 Implementation Overview xxvii
8.2 Mobile Application xxvii
8.3 Backend API xxvii
8.4 Document Processing xxvii
8.5 Chunking xxviii
8.6 Vector Storage xxviii
8.7 Search Implementation xxviii
8.8 RAG Implementation xxviii

9 Test Plans, Results and Analysis **xxix**
9.1 Testing Strategy xxix
9.2 Test Cases xxix
9.3 Performance Metrics xxx
9.3.1 Precision xxx
9.3.2 Recall xxx
9.3.3 Response Time xxx
9.3.4 Top-K Accuracy xxx
9.4 Expected Analysis xxx

10 Conclusion **xxxi**
10.1 Project Benefits xxxi
10.2 Future Scope xxxi

11 References **xxxiii**

A NPTEL Certificate **xxxv**

B Prototypes **xxxvi**
B.1 Prototype Overview xxxvi
B.2 Prototype Workflow xxxvi
B.3 Prototype Screenshots xxxvii

C Paper Publications **xxxviii**

C.1 Publication Details xxxviii

Abstract

The project aims to develop an intelligent multimodal information retrieval and knowledge management system that enables users to search and interact with personal documents using their meaning and content rather than only filenames. The system addresses the limitations of conventional file searching by converting uploaded text, images and documents into semantic vector representations. A vector database is used to perform similarity-based retrieval between natural-language queries and stored document embeddings. The system also integrates Retrieval-Augmented Generation (RAG) to enable users to ask questions and obtain context-aware answers from individual documents. The proposed solution is expected to improve information accessibility, semantic search accuracy and document interaction while reducing the time required to locate relevant information across large collections of multimedia files.

1 Introduction

1.1 Background

The rapid growth of digital information has resulted in users storing large numbers of documents, images, PDFs, scanned notes, presentations and other forms of multimedia content on personal devices and cloud platforms. Conventional file-management systems generally rely on filenames, directory structures, metadata and manually assigned tags for retrieving such information.

Although filename-based search is useful for files with descriptive names, it becomes inefficient when filenames are automatically generated or when the information contained within a file is more meaningful than its name. For example, images captured using smartphones may be stored with names such as "IMG_20261002_123456.jpg". Searching for a photograph based on its actual visual content is difficult using traditional filename-based mechanisms.

Recent developments in artificial intelligence, natural language processing, computer vision and vector databases have enabled systems to represent the semantic meaning of information as numerical vectors. Such representations can be compared using similarity measures, allowing users to search information using natural language descriptions rather than exact filenames.

The proposed project, **Intelligent Information Retrieval and Knowledge Management using Multimodal AI**, aims to apply these concepts to personal document and multimedia retrieval.

1.2 Motivation

The primary motivation behind this project is the difficulty users face when attempting to locate information distributed across a large collection of files.

Consider a user who has hundreds of photographs, PDFs and documents. Instead of remembering the filename, the user may remember only the content of the required file. For example:

- "Find the photo containing a red car."
- "Find the document related to machine learning algorithms."
- "Find all files containing the phrase semantic search."
- "Find the PDF where I discussed vector databases."
- "What does this particular document say about RAG?"

A semantic information retrieval system can address these use cases by understanding the content and meaning of the stored information.

1.3 Problem Statement

Traditional file-search systems primarily depend on filenames, folder structures and metadata. Such mechanisms do not adequately support semantic retrieval from heterogeneous multimedia content.

The problem addressed by this project is the development of a system that can:

1. index heterogeneous documents using semantic representations;
2. retrieve relevant documents based on natural-language queries;
3. search documents based on their content rather than only filenames;
4. support multimodal information such as text, PDFs and images;
5. allow users to interact conversationally with individual documents.

1.4 Technical Domain

The project falls within the following technical domains:

- Artificial Intelligence
- Natural Language Processing
- Information Retrieval
- Machine Learning
- Computer Vision
- Generative AI
- Retrieval-Augmented Generation
- Vector Databases
- Mobile Application Development
- Cloud Computing

1.5 Business Domain

The proposed system belongs to the broader domain of:

- Digital document management
- Knowledge management
- Personal information management
- Enterprise information retrieval
- Intelligent productivity software

1.6 Glossary

Table 1.1: Glossary of Important Terms

Term	Definition
Embedding	A numerical vector representation of the semantic characteristics of data.
Semantic Search	Searching information according to meaning rather than exact keyword matching.
Vector Database	A database optimized for storing and retrieving high-dimensional vectors using similarity measures.
RAG	Retrieval-Augmented Generation, a technique that retrieves relevant information before generating an answer.
Multimodal AI	Artificial intelligence systems capable of processing multiple types of information such as text and images.
OCR	Optical Character Recognition, used to extract text from images and scanned documents.
Top-K Retrieval	Retrieval of the K most relevant results according to a similarity measure.
Chunking	Dividing large documents into smaller pieces for processing and retrieval.
LLM	Large Language Model capable of understanding and generating natural language.
Cosine Similarity	A mathematical measure used to determine the similarity between two vectors.

2 Related Studies

2.1 Introduction

The proposed system combines concepts from information retrieval, natural language processing, computer vision, vector databases and generative artificial intelligence. Traditional information retrieval systems commonly use lexical approaches where documents and queries are compared using exact or statistically related terms.

Modern semantic retrieval systems use neural embeddings to represent queries and documents in a shared vector space. This enables retrieval based on semantic similarity even when the exact words used in the query and document differ.

2.2 Traditional Information Retrieval

Traditional information retrieval techniques include Boolean retrieval, TF-IDF and probabilistic retrieval models. These techniques generally operate on textual terms and their statistical importance within documents.

Although these methods remain useful, lexical matching can fail when semantically related information is expressed using different terminology.

2.3 Neural Embeddings

Neural embedding techniques represent text as dense numerical vectors. Semantically similar pieces of information tend to have vectors that are close to each other in the embedding space.

Sentence-BERT demonstrated the usefulness of transformer-based models for producing semantically meaningful sentence embeddings [2].

2.4 Transformer Architecture

Transformer-based architectures have significantly influenced natural language processing. The Transformer architecture introduced self-attention mechanisms that enable models to process relationships between tokens efficiently [1].

Large language models built upon transformer architectures have subsequently enabled increasingly sophisticated semantic understanding and text generation capabilities.

2.5 Vector Databases

Vector databases provide specialized mechanisms for storing high-dimensional embeddings and performing approximate nearest-neighbor searches. They are particularly relevant to semantic search systems where documents are represented by numerical vectors.

The proposed project uses this general architecture to retrieve documents according to

semantic similarity.

2.6 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with generative language models. Instead of relying entirely on information stored within model parameters, relevant external information can be retrieved and provided as context to the generation model.

Lewis et al. introduced a retrieval-augmented generation framework that combines a parametric generator with a non-parametric memory [3].

2.7 Multimodal Information Retrieval

Multimodal retrieval extends semantic retrieval beyond text. Images can be represented through visual embeddings while textual information can be represented through language embeddings. Multimodal representation models such as CLIP demonstrated the ability to learn relationships between images and natural-language descriptions [4].

This project applies similar principles to a personal document-management context.

3 Problem Definition and Preliminaries

3.1 Problem Definition

The project addresses the problem of retrieving documents and multimedia files based on their semantic content rather than relying exclusively on filenames.

The system accepts user-uploaded content, extracts relevant information, creates embeddings and stores these embeddings alongside metadata. When the user submits a natural-language query, the system generates a query embedding and searches the vector database for semantically similar documents.

3.2 Scope

The scope of the project includes:

- Uploading documents and images.
- Processing textual and visual content.
- Extracting text from suitable documents.
- Generating semantic embeddings.
- Storing embeddings in a vector database.
- Performing semantic similarity search.
- Returning top-K relevant documents.
- Conversational interaction with individual documents.
- Retrieval-Augmented Generation for document question answering.

3.3 Project Boundaries

The initial prototype is designed primarily for personal or controlled document collections. It does not attempt to replace enterprise-scale document management systems.

The project does not guarantee perfect semantic interpretation of every image or document. Retrieval quality depends on the embedding model, OCR quality, document quality and the characteristics of the stored content.

3.4 Preliminaries

The project requires understanding of:

- Natural Language Processing

- Machine Learning
- Deep Learning
- Transformer models
- Embeddings
- Vector similarity
- Vector databases
- Retrieval-Augmented Generation
- Mobile application development
- Backend API development

3.5 Similarity Calculation

One possible similarity measure is cosine similarity.

For two vectors A and B, cosine similarity is defined as:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{3.1}$$

A higher cosine similarity indicates greater directional similarity between the vectors.

4 Proposed Solution

4.1 Overview

The proposed system is an intelligent multimodal information retrieval and knowledge-management platform.

The system allows users to upload documents and multimedia files and later retrieve them using natural-language queries.

Instead of relying solely on filenames, the system creates semantic representations of uploaded content.

4.2 System Workflow

The overall workflow consists of two major pipelines:

1. Document ingestion and indexing.
2. Query processing and retrieval.

4.3 Document Ingestion Pipeline

When a user uploads a document, the system performs the following operations:

1. File upload.
2. File-type identification.
3. Content extraction.
4. OCR when required.
5. Text cleaning and preprocessing.
6. Document chunking.
7. Embedding generation.
8. Vector database insertion.
9. Metadata storage.

4.4 Semantic Search Pipeline

When the user submits a query:

1. The query is received by the backend.

2. The query is converted into an embedding.
3. The embedding is compared against indexed document embeddings.
4. Similarity scores are calculated.
5. The highest-ranked documents are selected.
6. The relevant documents are returned to the mobile application.

4.5 Document Chat Pipeline

The document-chat feature uses a Retrieval-Augmented Generation architecture.

The system retrieves relevant document chunks based on the user's question and supplies these chunks as context to the language model.

The generated response is therefore grounded in retrieved document content.

4.6 Major Contributions

The proposed system combines several capabilities within a single application:

- Semantic document search.
- Content-based image search.
- Multimodal indexing.
- Vector-based retrieval.
- Document-specific conversational interaction.
- RAG-based question answering.
- Centralized personal knowledge retrieval.

5 Project Planning

5.1 Development Methodology

The project follows an iterative development methodology. The system is divided into independent modules that can be implemented, tested and refined separately.

5.2 Major Project Phases

Table 5.1: Project Development Phases

Phase	Activity	Expected Outcome
1	Problem identification	Defined problem statement
2	Literature survey	Understanding existing techniques
3	Requirement analysis	Functional and non-functional requirements
4	System design	Architecture and database design
5	Prototype development	Initial working modules
6	AI integration	Embedding and RAG pipeline
7	Application development	Mobile application integration
8	Testing	Functional and performance evaluation
9	Documentation	Final report and presentation

5.3 Software Life Cycle Model

An iterative and incremental software development approach is suitable for this project because the system contains multiple AI-based components whose performance may require experimentation and refinement.

5.4 Estimated Cost

The project can initially be implemented using open-source software frameworks and development tools.

Potential costs may include:

- Cloud infrastructure.
- Vector database hosting.
- Large language model API usage.

- Storage.
- Mobile application deployment.

6 Requirement Analysis

6.1 Functional Requirements

6.1.1 User Registration and Authentication

The system should provide mechanisms for users to securely access their document collections.

6.1.2 Document Upload

Users should be able to upload supported document and image formats.

6.1.3 Document Processing

The system should extract useful information from uploaded content.

6.1.4 Embedding Generation

The system should generate semantic embeddings for indexed content.

6.1.5 Semantic Search

Users should be able to search documents using natural-language queries.

6.1.6 Document Chat

Users should be able to ask questions about a selected document.

6.1.7 Search Result Ranking

The system should return the most semantically relevant results.

6.2 Non-Functional Requirements

The system should satisfy the following:

- Usability
- Scalability
- Security
- Reliability
- Maintainability
- Performance
- Privacy

6.3 Requirement Matrix

Table 6.1: Requirement Matrix

ID	Requirement	Description	Priority
FR-01	User Authentication	Users should be able to securely access their account.	High
FR-02	File Upload	Users should be able to upload supported files.	High
FR-03	Content Extraction	System should extract textual and visual information.	High
FR-04	Embedding Generation	System should generate semantic embeddings.	High
FR-05	Vector Storage	Embeddings should be stored in a vector database.	High
FR-06	Semantic Search	Users should search using natural language.	High
FR-07	Top-K Retrieval	System should return relevant documents.	High
FR-08	Document Chat	Users should interact with individual documents.	High
FR-09	RAG	System should retrieve context before generating answers.	High
FR-10	Metadata Management	System should maintain file metadata.	Medium
NFR-01	Security	User data should be protected.	High
NFR-02	Performance	Search should provide responses within an acceptable time.	High
NFR-03	Scalability	Architecture should support increasing document collections.	Medium
NFR-04	Usability	Interface should be easy to use.	High

6.4 Requirement Elaboration

6.4.1 FR-01 User Authentication

The application shall provide authentication functionality so that each user can access only their own indexed document collection.

6.4.2 FR-02 File Upload

The application shall allow users to upload supported file formats. The uploaded file shall be associated with the corresponding user account.

6.4.3 FR-03 Content Extraction

The backend shall identify the uploaded file type and invoke the appropriate processing pipeline.

6.4.4 FR-04 Embedding Generation

The system shall convert extracted information into vector representations using an appropriate embedding model.

6.4.5 FR-05 Vector Storage

The generated embeddings shall be stored with appropriate metadata.

6.4.6 FR-06 Semantic Search

The application shall accept natural-language queries and return semantically relevant documents.

6.4.7 FR-07 Top-K Retrieval

The retrieval system shall rank indexed content according to a similarity score and return the top-K results.

6.4.8 FR-08 Document Chat

The application shall allow users to select a document and ask natural-language questions regarding its content.

6.4.9 FR-09 Retrieval-Augmented Generation

The system shall retrieve relevant document chunks and provide them to the generation model as contextual information.

6.4.10 NFR-01 Security

User documents and associated embeddings should be isolated between users.

7 Design

7.1 Technical Environment

7.1.1 Hardware Requirements

A development system with the following minimum configuration is recommended:

Table 7.1: Minimum Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 / equivalent or above
RAM	8 GB minimum; 16 GB recommended
Storage	At least 20 GB available storage
Internet	Stable broadband connection
Mobile Device	Android/iOS device for application testing

7.1.2 Software Environment

Table 7.2: Software Environment

Component	Technology
Operating System	Windows / Linux / macOS
Programming Language	Python / JavaScript / TypeScript
Mobile Framework	React Native / Flutter
Backend	FastAPI / Node.js
Database	PostgreSQL / equivalent
Vector Database	ChromaDB / FAISS / Pinecone / equivalent
AI Framework	PyTorch / Hugging Face Transformers
Development Environment	VS Code / PyCharm
Version Control	Git

7.2 Detailed Design

7.2.1 System Architecture

The system consists of the following major layers:

1. Mobile Application Layer

2. Backend API Layer
3. Document Processing Layer
4. Embedding Layer
5. Vector Database Layer
6. Retrieval Layer
7. RAG and LLM Layer



Figure 7.1: High-Level System Architecture

7.2.2 Document Ingestion Module

The document ingestion module is responsible for receiving files and preparing them for indexing.

The module performs:

1. File validation.
2. File-type identification.
3. Text extraction.
4. OCR where necessary.
5. Image processing.
6. Text normalization.
7. Chunk generation.
8. Embedding generation.
9. Vector storage.

7.2.3 Embedding Module

The embedding module converts processed content into dense numerical representations.

For a document D, the embedding function can be represented as:

$$E_D = f(D) \tag{7.1}$$

where f represents the embedding model and E_D represents the resulting vector.

7.2.4 Semantic Search Module

For a query Q, the system generates:

$$E_Q = f(Q) \tag{7.2}$$

The system then compares E_Q against document vectors stored in the vector database.

7.2.5 Retrieval Module

The retrieval module performs nearest-neighbor search and returns the top-K documents or chunks.

7.2.6 RAG Module

The RAG module follows the following process:

1. Receive user question.
2. Generate question embedding.
3. Retrieve relevant chunks.
4. Construct contextual prompt.
5. Send context and question to the language model.
6. Return generated answer.

7.2.7 Database Design

The conceptual database contains entities such as:

- User
- Document
- Document Chunk
- Embedding
- Search Query
- Chat Session
- Chat Message

Table 7.3: Document Entity

Attribute	**Type**	**Description**
document_id	UUID	Unique document identifier
user_id	UUID	Owner of document
filename	String	Original filename
file_type	String	MIME/file type
file_path	String	Storage location
created_at	Timestamp	Upload timestamp

8 Implementation

8.1 Implementation Overview

The implementation phase converts the proposed architecture into a functional prototype. The major implementation components are the mobile application, backend API, document-processing pipeline, embedding service, vector database and RAG pipeline.

8.2 Mobile Application

The mobile application provides the user interface for:

- Authentication.
- File upload.
- Search.
- Search-result browsing.
- Document preview.
- Document chat.

8.3 Backend API

The backend exposes APIs responsible for:

- User authentication.
- File upload.
- Document processing.
- Search queries.
- Retrieval.
- Chat requests.

8.4 Document Processing

The document-processing service determines how a particular file should be processed.

For example:

- PDF $\rightarrow$ text extraction.
- Image $\rightarrow$ OCR and visual processing.

- Text file → direct text processing.
- Scanned document → OCR.

8.5 Chunking

Large documents are divided into smaller chunks before embedding.

Chunking improves retrieval granularity because the system can retrieve the specific portion of a document relevant to a query rather than retrieving the complete document.

8.6 Vector Storage

Each processed chunk is represented by an embedding vector.

The vector is stored along with metadata such as:

- Document ID
- User ID
- Chunk ID
- Original filename
- Page number
- Content type

8.7 Search Implementation

A natural-language query is converted into an embedding.

The vector database then performs similarity search.

The system retrieves the top-K candidates according to the similarity metric.

8.8 RAG Implementation

The document-chat implementation uses retrieved chunks as context.

A simplified conceptual prompt is:

```
Context:
[Retrieved document chunks]

Question:
[User question]

Answer using the supplied context.
```

The generated response is returned to the mobile application.

9 Test Plans, Results and Analysis

9.1 Testing Strategy

The system should be tested at multiple levels:

- Unit testing
- Integration testing
- Functional testing
- Retrieval testing
- Performance testing
- User interface testing

9.2 Test Cases

Table 9.1: Test Cases

ID	Test Case	Expected Result	Status
UP-01	Upload valid PDF	PDF should be successfully uploaded and indexed.	Pass/Fail
UP-02	Upload image	Image should be processed and indexed.	Pass/Fail
SR-01	Search by semantic query	Relevant documents should be returned.	Pass/Fail
SR-02	Search using related terminology	Semantically related documents should be retrieved.	Pass/Fail
SR-03	Search across multiple documents	Relevant documents should be ranked.	Pass/Fail
CH-01	Ask question about document	System should provide context-aware answer.	Pass/Fail
CH-02	Ask unrelated question	System should avoid unsupported document-specific claims.	Pass/Fail

PR-01	Measure search latency	Response time should remain within target.	Pass/Fail

9.3 Performance Metrics

The following metrics can be used to evaluate the system.

9.3.1 Precision

Precision measures the proportion of retrieved results that are relevant.

$$Precision = \frac{Relevant\ Retrieved\ Documents}{Total\ Retrieved\ Documents} \tag{9.1}$$

9.3.2 Recall

Recall measures the proportion of relevant documents that were successfully retrieved.

$$Recall = \frac{Relevant\ Retrieved\ Documents}{Total\ Relevant\ Documents} \tag{9.2}$$

9.3.3 Response Time

Response time measures the duration between submission of a search request and receipt of the search results.

9.3.4 Top-K Accuracy

Top-K evaluation can be used to determine whether the expected relevant document appears within the first K retrieved results.

9.4 Expected Analysis

The evaluation should examine:

- Semantic retrieval quality.
- Search latency.
- Retrieval precision.
- Retrieval recall.
- RAG answer relevance.
- OCR quality.
- Performance across different document types.

Actual numerical results should be inserted after conducting experiments on the implemented prototype.

10 Conclusion

The proposed project, **Intelligent Information Retrieval and Knowledge Management using Multimodal AI**, addresses the limitations of conventional filename-based document retrieval by introducing semantic and content-based information retrieval.

The system combines document processing, embeddings, vector similarity search and Retrieval-Augmented Generation into a unified architecture. Users can search their personal document collections using natural-language descriptions and can interact with individual documents through a conversational interface.

The proposed system has potential applications in personal knowledge management, academic document management, enterprise information retrieval and intelligent productivity systems.

10.1 Project Benefits

The major benefits of the proposed system include:

- Content-based document retrieval.
- Semantic rather than exact keyword search.
- Reduced dependence on meaningful filenames.
- Search across heterogeneous documents.
- Natural-language interaction with documents.
- Improved accessibility of personal information.
- Potential integration with multimodal AI systems.

10.2 Future Scope

Several improvements can be explored in future versions:

1. Improved multimodal embeddings.
2. Support for video and audio search.
3. Speaker-aware audio indexing.
4. Handwriting recognition.
5. Cross-language semantic search.
6. Automatic document categorization.

7. Duplicate document detection.
8. Personalized retrieval.
9. On-device AI processing.
10. Privacy-preserving local inference.
11. Advanced citation and source-grounding mechanisms.
12. Federated or distributed knowledge management.

11 References

Bibliography

[1] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.

[4] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning*, 2021.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of NAACL-HLT*, 2019.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

A NPTEL Certificate

A digital copy of the relevant NPTEL course certificate shall be inserted in this section.

NPTEL certificate to be inserted here.

B Prototypes

B.1 Prototype Overview

The prototype demonstrates the core functionality of the proposed system.

The prototype should include the following screens:

1. Login / Registration
2. Home screen
3. Document upload
4. Document library
5. Semantic search
6. Search results
7. Document preview
8. Chat with document
9. Settings

B.2 Prototype Workflow

The prototype workflow is:

1. User opens the mobile application.
2. User uploads a document or image.
3. Backend processes the content.
4. Embeddings are generated.
5. Embeddings are stored in the vector database.
6. User submits a semantic query.
7. Relevant documents are retrieved.
8. User selects a document.
9. User asks questions about the document.
10. RAG pipeline generates the response.

B.3 Prototype Screenshots

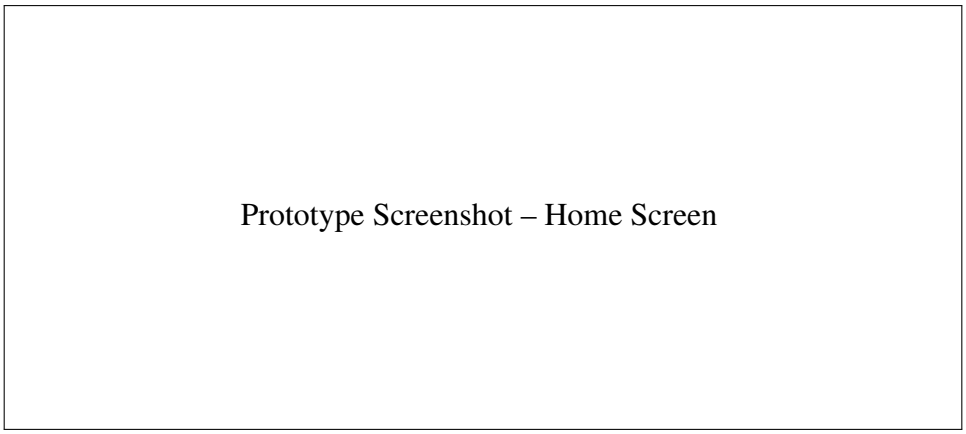


Figure B.1: Prototype Home Screen

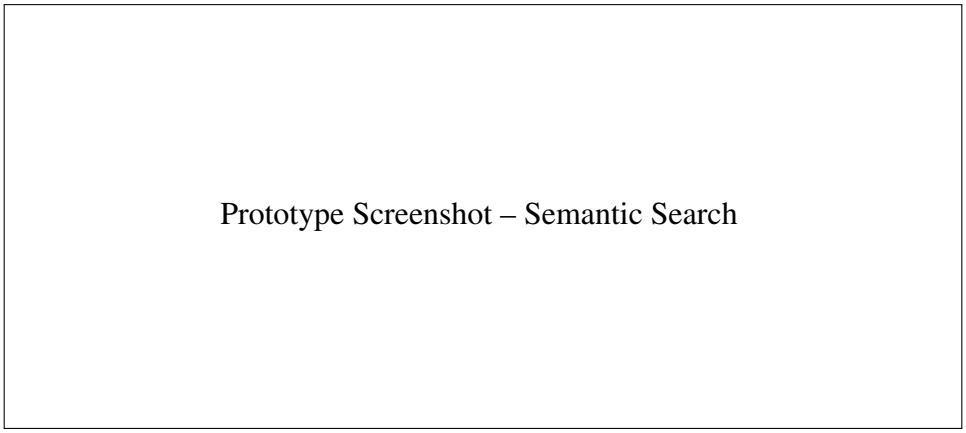


Figure B.2: Prototype Semantic Search Screen

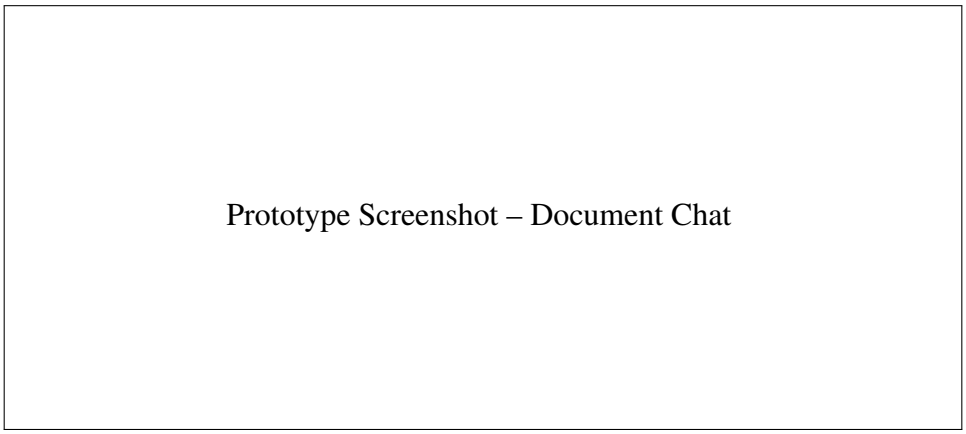


Figure B.3: Prototype Document Chat Screen

C Paper Publications

If a paper based on this project is published or accepted at a recognized journal or conference, the relevant publication details and documentation should be included in this appendix.

C.1 Publication Details

Table C.1: Paper Publication Details

Field	**Details**
Paper Title	[To be added]
Authors	Tapaban Ray, Saptaparna Pati, Swarnava Maity, Ahyan Kamal
Journal / Conference	[To be added]
Publication Date	[To be added]
DOI / Paper ID	[To be added]